

Toma de Decisiones Secuenciales y Aprendizaje por Refuerzo

Daniela Pucci de Farias

Universidad Torcuato Di Tella

Modulo 3: Aprendizaje por Refuerzo

En el módulo anterior, estudiamos el marco de MDPs para resolver problemas de toma de decisiones secuenciales. La clave fue la **Ecuación de Bellman**:

$$V^*(s) = \max_{a \in A} \left\{ r(s, a) + \alpha \sum_{s'} P(s, a, s') V^*(s') \right\}$$

- **Iteración de Valor:** Actualiza valores hasta converger.
- **Iteración de Política:** Alterna entre evaluar V^u (sistema lineal) y mejorar u (*greedy step*).

¿Por qué no siempre podemos usar MDPs clásicos?

El paradigma de MDP es potente, pero enfrentamos barreras críticas al aplicarlo en la industria:

- **Incertidumbre en los Parámetros:** En un e-commerce (precificación), no conocemos la probabilidad real de que un cliente acepte un precio.
- **Observabilidad Parcial:** En la gestión de relaciones con clientes (CRM), el «estado emocional» del cliente no se ve. Solo tenemos indicadores ruidosos (clics, quejas).
- **La Maldición de la Dimensionalidad:** Si gestionamos un inventario de 100 productos, ¿cómo será el espacio de estados? ¿Cuánto tiempo y espacio se necesitará para calcular y almacenar V^* ?

El quiebre del modelo

Cuando las reglas (P, r) son desconocidas o el espacio es inabarcable, la Programación Dinámica tradicional falla.

Desafíos y sus Soluciones Modernas

1. Información Parcial

No conocemos $P(s'|s, a)$ ni $r(s, a)$. **Solución: Aprendizaje**

El agente debe interactuar con el entorno y aprender de la experiencia (**RL**).

2. Estados Ocultos

El estado real no es visible. **Solución: POMDPs**

Actuar en el espacio de probabilidades de estados.

3. Complejidad Masiva

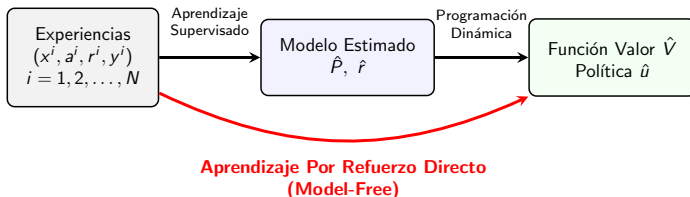
Millones de estados o acciones continuas. **Solución: Generalización**

En lugar de tablas, empleamos arquitecturas de aproximación (como redes neuronales) para representar las funciones de valor y/o política.

Desafío: Incertidumbre y Datos

- Hasta ahora, dependíamos de P y r . ¿Qué sucede cuando las reglas del mundo son desconocidas o demasiado complejas para modelar?
- **La Propuesta:** »Aprender« la política óptima directamente a partir de la **interacción** con el ambiente.
- **¿Con qué datos contamos?** En lugar de matrices, usamos experiencias:
 - **Trayectorias:** Secuencias temporales de estados y acciones.
 - Única (horizonte largo): $(x_0, a_0, r_0, x_1, a_1, r_1, \dots)$
 - Múltiples (episodios): $\{(x_0^i, a_0^i, r_0^i, \dots, x_T^i)\}_{i=1}^M$
 - **Tuplas (Transiciones):** Conjuntos de experiencias aisladas (x, a, r, y) .

Model-Based vs. Model-Free RL



- **Enfoque Indirecto (Model-Based):** Construimos un simulador mental del mundo.
 - **Frecuentista:** $\hat{P}(x, y) = \frac{|I_{x,y}|}{|I_x|}$. Estimamos probabilidades contando transiciones.
 - **Paramétrico:** Estimamos leyes físicas o económicas (ej: volatilidad σ).
- **Enfoque Directo (Model-Free):** Saltamos el paso del modelo.
 - Aprendemos $\hat{V}$ o $\hat{u}$ directamente de los errores de predicción.

¿Cuándo es atractivo el enfoque Model-Based?

Construir un **Modelo del Ambiente** suele ser la mejor opción cuando el sistema posee una estructura que podemos explotar:

- **Estructura y Ejemplos:**

- **Modelos Paramétricos:** Ecuaciones de la física o econometría (ej. leyes de Newton, Black-Scholes). Pocos parámetros para estimar.
- **Digital Twins:** Réplicas digitales de alta fidelidad de activos físicos (ej. una turbina eólica o una red eléctrica).
- **Simuladores Estocásticos:** Modelos de eventos discretos o Monte Carlo para logística y colas.

- **Sistemas Críticos (Seguridad):** En medicina o energía, el costo de "fallar" es infinito. El modelo permite **testeo offline** y simulaciones de escenarios catastróficos.

- **Eficiencia de Muestra:** La ventaja nace de la **estructura**. El modelo nos permite inferir estados nunca visitados generalizando información de otros estados mediante la lógica del sistema.

Aprendizaje por Refuerzo Directo (Model-Free)

- **Independencia del Modelo:** Aprendemos la política **mientras se actúa**, sin fijar la dinámica del ambiente.
 - *Dinámicas cambiantes:* Ideal para entornos no estacionarios (mercados, recomendación a usuarios).
- **Robustez:** Evitamos el riesgo de **sesgo del modelo** (*model bias*) al no imponer estructuras paramétricas rígidas.
- **Interacción con »Cajas Negras«:** Esencial cuando el modelo es un **simulador** complejo.
 - No accedemos a P o r ; solo observamos transiciones.
 - *Ejemplos:* Videojuegos, procesos industriales, clima.
- **Escalabilidad:** Permite el uso de *Function Approximation* (redes neuronales) para manejar espacios de estados donde la Programación Dinámica tradicional colapsa.

En resumen

Model-Free es preferible cuando la complejidad del sistema impide su modelado o cuando la fidelidad a la experiencia real prima sobre la eficiencia de datos.

Comparativa: Model-Based vs. Model-Free

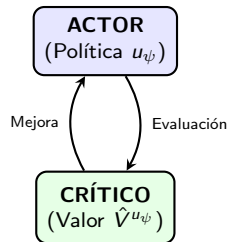
Característica	Model-Based	Model-Free
<i>Fortaleza</i>	Eficiencia de datos (si hay estructura).	Robustez ante modelos mal especificados.
<i>Punto Débil</i>	Riesgo de sesgo del modelo (<i>Model Bias</i>).	Requiere una cantidad masiva de datos.
<i>Escalabilidad</i>	Complejo en espacios continuos/masivos.	Naturalmente apto para combinar con arquitecturas de aproximación.

La Confluencia: RL como Solución de Bellman

Incluso con un modelo perfecto, la **maldición de la dimensionalidad** puede impedir resolver Bellman de forma exacta. En esos casos, usamos algoritmos de RL no para aprender el mundo, sino como un método de **programación dinámica aproximada**.

Aprendizaje por Refuerzo: El Mapa de Ruta

- 1 **Valor (Value-based):** Aprendemos $V^*(x)$ o $Q^*(x, a)$. La política es una consecuencia.
- 2 **Política (Policy Search):** Utilizamos una clase paramétrica de políticas $\hat{u}_\psi$ y optimizamos sobre ψ directamente.
- 3 **Actor-Crítico:** Híbrido que entrena ambos en simultáneo.



Foco del Curso

Nos concentraremos en **Métodos de Valor**, la base para entender cómo resolver Bellman.^a partir de datos.

- **Optimización Directa de Política (Policy Search):**

- Se define $\hat{u}(x) = u(x, \psi)$ (ej: comprar si stock $< \psi_1$).
- Se ajusta ψ usando *gradiente estocástico* para maximizar el retorno esperado.
- **Pros:** Natural para acciones continuas; **Contras:** Alta varianza en el aprendizaje.

- **Métodos Actor-Crítico:**

- El **Actor** propone la acción y el **Crítico** (basado en valor) evalúa si fue buena.
- Es el estándar en RL moderno (Deep RL) porque combina la estabilidad del valor con la flexibilidad de la política.

¿Por qué mencionarlos?

Aunque no los aplicaremos, entender que existen permite saber cuándo Q-Learning es suficiente y cuándo se requiere un esquema más complejo.

Nuestra Ruta: Del Valor a la Práctica

En este módulo, profundizaremos en los métodos basados en valor:

- **TD-Learning (Temporal Difference):** Cómo aprender de la experiencia paso a paso sin esperar al final de la trayectoria.
- **Q-Learning:** El algoritmo »off-policy« por excelencia para aprender la acción óptima de forma directa.
- **Arquitecturas de Aproximación:**
 - ¿Qué pasa cuando el estado es continuo o gigante?
 - Veremos **Longstaff-Schwartz** como un caso de éxito de aproximación en finanzas (opciones americanas).

Idea Clave: Usaremos RL como un *solver* iterativo de la Ecuación de Bellman.

Estimación de La Función Valor

- La función valor es definida implícitamente por la ecuación de Bellman:

$$V^*(x) = \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V^*(y) \right\}$$

- Cómo estimarla a partir de datos?
- Empezamos con el caso de una política fija
 - proceso markoviano con recompensa (MRP)

Estimación de La Función Valor: Monte Carlo

- Con una política fija u , el valor es el retorno esperado:

$$V_u(x) = \mathbb{E} \left[\sum_{k=0}^{\infty} \alpha^k r_{t+k} \mid x_t = x \right]$$

- A partir de una trayectoria $(x_0, r_0, x_1, r_1, \dots)$, calculamos el **retorno acumulado** R_t para cada paso:

$$R_t = r_t + \alpha r_{t+1} + \alpha^2 r_{t+2} + \dots = \sum_{k=0}^{\infty} \alpha^k r_{t+k}$$

- **Estimador MC:** Para cada estado x , promediamos los retornos observados tras visitarlo:

$$\hat{V}(x) = \frac{\sum_{t \in T_x} R_t}{|T_x|}$$

donde T_x son los instantes de tiempo en que se visitó el estado x .

Del Retorno al Bootstrapping

- **Limitaciones de Monte Carlo:**

- Solo se puede actualizar al **final** de la trayectoria (problema en episodios largos o infinitos).
- Alta varianza: cada retorno R_t es la suma de muchas variables aleatorias.

- **La Idea del Bootstrapping:** Relacionar el valor actual con el valor del estado siguiente.

$$R_t = r_t + \underbrace{\alpha(r_{t+1} + \alpha r_{t+2} + \dots)}_{R_{t+1}}$$
$$\approx r_t + \alpha \hat{V}(x_{t+1})$$

- **Actualización Incremental:** Podemos actualizar $\hat{V}(x_t)$ inmediatamente al observar la transición (x_t, r_t, x_{t+1}) :

$$\hat{V}(x_t) \leftarrow (1 - \gamma_t) \hat{V}(x_t) + \gamma_t \underbrace{\left[r_t + \alpha \hat{V}(x_{t+1}) \right]}_{\text{Target TD}}$$

Algoritmo TD(0)

- 1 Inicializar $\hat{V}(x)$ arbitrariamente (ej. ceros).
- 2 Por cada paso en la experiencia (x, r, x') :

$$\hat{V}(x) \leftarrow \hat{V}(x) + \gamma \underbrace{\left[\overbrace{r + \alpha \hat{V}(x')}^{\text{Target}} - \hat{V}(x) \right]}_{\text{Diferencia Temporal (error de Bellman)}}$$

- **Visión Teórica:** TD-learning es la versión **estocástica** de la Iteración de Valor. En promedio, la actualización sigue la dirección de la ecuación de Bellman.
- **Ventaja Crítica:** No necesitamos trayectorias completas. Podemos aprender de fragmentos sueltos de experiencia o de transiciones individuales de estado.

De la Evaluación a la Optimización

Hemos visto que TD-Learning converge a V_u para un proceso con política fija (MRP). ¿Podemos extender esto para encontrar la política óptima V^* ?

- **La Extensión Natural:** En un MDP, el objetivo es satisfacer la ecuación de Bellman de optimalidad:

$$V^*(x) = \max_{a \in A} \mathbb{E}[r + \alpha V^*(y) \mid x, a]$$

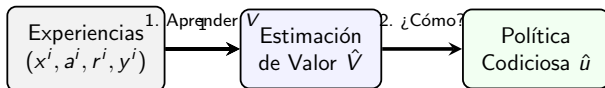
- **TD para Optimalidad:** Si aplicamos la misma lógica de *bootstrapping*, el "target" de nuestra actualización ahora incluye el operador de maximización:

$$\hat{V}(x) \leftarrow \hat{V}(x) + \gamma \left[\max_{a \in A} (r_a + \alpha \hat{V}(y)) - \hat{V}(x) \right]$$

- **Garantía Teórica:** Bajo condiciones de exploración suficientes, este esquema converge a la función valor óptima V^* .

Un Obstáculo en el Aprendizaje Directo

- TD-Learning nos permite obtener una estimativa de la la función valor $\hat{V}$. Con suficientes simulaciones, esa estimativa convergirá a V^* .



- **El Problema:** Para derivar la política desde $\hat{V}$ necesitamos:

$$\hat{u}(x) = \arg \max_{a \in A} \left\{ \underbrace{r_a(x) + \alpha \sum_y P_a(x, y) \hat{V}(y)}_{\text{¡Requiere el Modelo!}} \right\}$$

- **Conclusión:** Si no tenemos P ni r , conocer V^* , mismo perfectamente, no es suficiente para actuar.

La Solución: La Función Q

La función $Q^*(x, a)$ representa el valor de tomar la acción a en el estado x y seguir la política óptima de ahí en adelante.

- **Definición:**

$$Q^*(x, a) = r_a(x) + \alpha \sum_y P_a(x, y) V^*(y)$$

- **Relación con V^* :** Como $V^*(x) = \max_a Q^*(x, a)$, podemos reescribir Bellman:

$$Q^*(x, a) = r_a(x) + \alpha \sum_y P_a(x, y) \max_{a' \in A} Q^*(y, a')$$

- **La gran ventaja:** Con una aproximación $\hat{Q}$, la política es **directa**:

$$\hat{u}(x) = \arg \max_{a \in A} \hat{Q}(x, a)$$

¡No necesitamos conocer las probabilidades de transición P !

Algoritmo: Q-Learning

Q-Learning (Watkins, 1989)

- 1 Inicializar $\hat{Q}(x, a)$ arbitrariamente.
- 2 Por cada transición observada (x, a, r, y) :

$$\hat{Q}(x, a) \leftarrow \hat{Q}(x, a) + \gamma \underbrace{\left[\overbrace{r + \alpha \max_{a' \in A} \hat{Q}(y, a')}^{\text{Target (Bellman)}} - \hat{Q}(x, a) \right]}_{\text{Diferencia Temporal (TD)}}$$

- 3 Repetir hasta convergencia.
- **Intuición:** Si el término entre corchetes es positivo, la acción a resultó ser mejor de lo que esperábamos $\rightarrow$ aumentamos $\hat{Q}(x, a)$.
 - **Off-policy:** Podemos aprender de **cualquier** dato (incluso si la acción a fue tomada por una política mediocre o al azar).

Calibración y Desafíos Prácticos

La implementación exitosa de Q-Learning exige una sintonía fina. No basta con programar la fórmula; hay que gestionar cómo aprende el agente.

- **Tasa de Aprendizaje (γ):** Controla el balance entre la **memoria** (lo aprendido) y la **novedad** (la muestra actual). Determina la estabilidad frente al ruido.
- **Selección de la Acción:** Resuelve el dilema de **Exploración vs. Explotación**. ¿Usamos lo que ya funciona o buscamos algo mejor?
- **Desafíos de Ingeniería:** Incluso con parámetros óptimos, enfrentaremos problemas de sesgo, correlación y escasez de recompensas.

Lección de Experiencia

La mayoría de las fallas en proyectos de RL no están en el algoritmo, sino en una mala calibración de estos pilares.

Exploración: ¿Costo o Estrategia?

Para que $\hat{Q} \rightarrow Q^*$, el agente debe **explorar suficientemente** el espacio de acciones.

- **Eficiencia:** El azar puro garantiza convergencia, pero es computacionalmente lento.
- **Online Learning (En Vivo):** Dilema económico: explorar es costoso (pérdida de recompensa real).
- **Offline Learning (Simulador):** Problema de cobertura: explorar asegura iluminar regiones clave de la tabla Q .

La Política ϵ -greedy

Equilibrio base entre búsqueda y aprovechamiento:

$$a_t = \begin{cases} \arg \max_a Q(x, a) & \text{con prob. } 1 - \epsilon \text{ (Explotación)} \\ \text{acción aleatoria} & \text{con prob. } \epsilon \text{ (Exploración)} \end{cases}$$

ϵ -greedy explora a ciegas. Alternativas para problemas complejos:

- **Exploración de Boltzmann (Softmax):** Selección probabilística según el valor estimado:

$$P(a|x) \propto e^{Q(x,a)/\tau}$$

Evita acciones desastrosas al priorizar opciones “casi-óptimas”.

- **Optimismo (UCB / Initial Values):** Inicializar Q con valores muy altos. Fuerza la exploración inicial hasta que la realidad corrige las expectativas.
- **Seguro contra el cambio:** En entornos **no estacionarios**, mantener $\epsilon > 0$ permite detectar cambios estructurales en el sistema.

Tasa de Aprendizaje: Memoria vs. Ruido

Podemos reescribir la actualización de Q-Learning para ver cómo γ pondera la información. Por cada transición observada (x, a, r, y) :

$$\underbrace{\hat{Q}(x, a)}_{\text{Nuevo Valor}} \leftarrow (1 - \gamma) \underbrace{\hat{Q}(x, a)}_{\text{Valor Anterior}} + \gamma \underbrace{\left[r + \alpha \max_{a' \in A} \hat{Q}(y, a') \right]}_{\text{Nueva Muestra (Target)}}$$

- γ **alto (ej: 0.5):**

- *Ventaja:* El algoritmo incorpora información nueva rápidamente.
- *Riesgo:* Alta **varianza**. Si las muestras son ruidosas, Q oscilará sin converger.

- γ **bajo (ej: 0.01):**

- *Ventaja:* Estable ante el ruido; actúa como un filtro de media móvil.
- *Riesgo:* Aprendizaje muy lento (*estancamiento*).

En la práctica

Si el entorno es **no estacionario** (cambiante), nunca debemos llevar γ a cero, o el agente dejará de notar los cambios en el sistema.

Aplicar Q-Learning tabular a problemas reales suele fallar por tres motivos:

- ❶ **Escasez de Señal:** El agente no encuentra recompensas (sparse rewards).
- ❷ **Inestabilidad:** Los datos están correlacionados temporalmente.
- ❸ **Sobreestimación:** El ruido hace que el agente sea "demasiado optimista".

Las siguientes diapositivas presentan soluciones de ingeniería para estos retos.

- **Problema:** En algunas tareas, el agente recibe 0 casi siempre.
¿Examples?
- **Solución:** Diseñar recompensas intermedias para guiar el aprendizaje.
- **Peligro:** *Reward Hacking*. El agente puede encontrar un atajo para ganar puntos sin resolver la tarea (ej: un robot que da vueltas en círculos para ganar "puntos por movimiento").

El Problema: El operador máx es convexo. Por la **Desigualdad de Jensen**, el ruido inyecta un sesgo de sobreestimación:

$$\mathbb{E}[\max_a \hat{Q}(s, a)] \geq \max_a \mathbb{E}[\hat{Q}(s, a)]$$

- **Solución:** Desacoplar **selección** de **evaluación**.
- **Mecánica (Simétrica):** Mantener Q_1 y Q_2 . **Actualizar** Q_1 o Q_2 , aleatoriamente. Si actualizamos Q_1 :
 - 1 **Selección:** $a^* = \arg \max_a Q_1(y, a)$
 - 2 **Evaluación:** El target se calcula con la *otra* tabla:

$$Q_1 \leftarrow Q_1 + \gamma[r + \alpha Q_2(y, a^*) - Q_1]$$

Intuición de Ensamble

La »segunda opinión« de un estimador independiente elimina el optimismo sistemático del ruido.

- **Problema:** Los datos (x, a, r, y) en una trayectoria están altamente correlacionados. Esto hace que el aprendizaje sea inestable.
- **Solución:** Guardar las experiencias en un *buffer* (memoria) y entrenar extrayendo mini-batches al azar.
- **Beneficio:** Rompe la correlación temporal y permite aprender varias veces de experiencias valiosas.

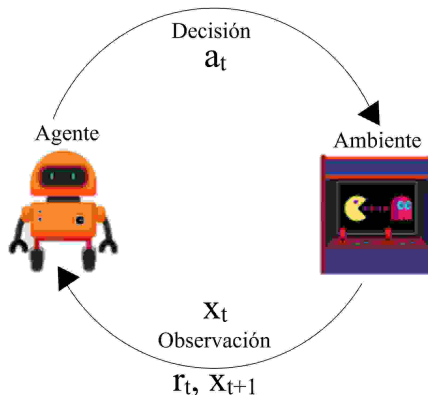
Conclusiones: De la Teoría a la Implementación

El paso de la programación dinámica al aprendizaje por refuerzo implica gestionar la incertidumbre y el ruido.

- **El poder del Bootstrapping:** TD-Learning permite aprender sin un modelo del entorno, actualizando estimaciones basadas en otras estimaciones.
- **Calibración como Prioridad:** El éxito no depende solo del algoritmo, sino del equilibrio entre **exploración** (cobertura) y **tasa de aprendizaje** (estabilidad vs. velocidad).
- **Pensamiento de Diseño:** Como científicos, nuestro rol es diseñar incentivos (recompensas) y estructuras que permitan que la razón sirva a la existencia del agente en entornos complejos.

Próximamente: La maldición de la dimensionalidad y aproximación de funciones.

Revisión - MDPs y Aprendizaje por Refuerzo



- **Agente:**

- Función valor $V(x)$ o $Q(x, a)$
- Política $u(x)$

- **Ambiente:**

- Recompensa $r_a(x)$
- Transición $P_a(x, y)$

- La función valor óptima satisface:

$$\begin{aligned} V^*(x) &= \max_{a \in A} \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V^*(y) \right\} \\ Q^*(x, a) &= r_a(x) + \alpha \sum_y P_a(x, y) \max_{a' \in A} Q^*(y, a') \end{aligned}$$

- Estrategias de solución:

Soluciones Exactas	Aprendizaje (Model-Free)
Iteración de valor	Monte Carlo (Política fija)
Iteración de política	TD-Learning / Q-Learning

- En ambos casos, el objetivo es derivar la política óptima $u^*(x)$.

Algoritmo

- 1 Inicializar tabla $\hat{Q}$ y $t = 1$.
- 2 Seleccionar acción a^i (ej: ϵ -greedy).
- 3 Observar (x^i, a^i, r^i, y^i) y actualizar:

$$\hat{Q}(x^i, a^i) \leftarrow \hat{Q}(x^i, a^i) + \gamma_i \left[r^i + \alpha \max_a \hat{Q}(y^i, a) - \hat{Q}(x^i, a^i) \right]$$

- 4 Hacer $t = t + 1$ y volver al paso 2.

- **Learning rate (γ_i):** Controla la convergencia. Ej: $\gamma_i = 1/N(x, a)$.
- **Limitación fundamental:** Cada par (x, a) es una celda independiente en la tabla.

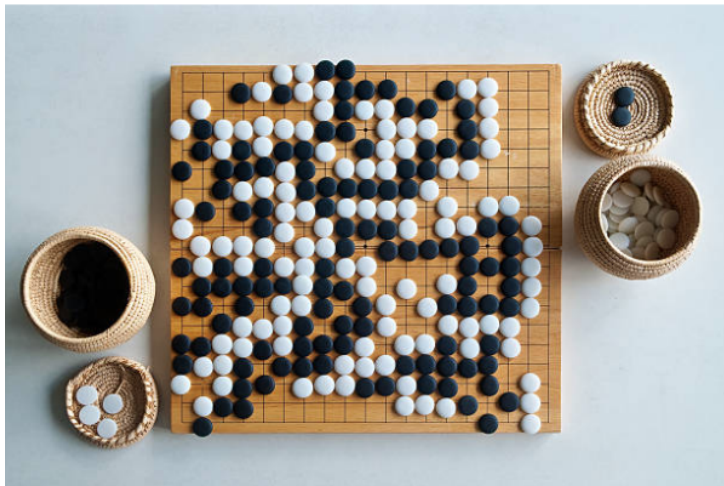
La “Maldición de la Dimensionalidad”

- El enfoque tabular requiere almacenar V^* o Q^* en memoria.
- Imaginemos una tienda que debe decidir cuánto pedir de cada producto para minimizar costos de almacenamiento y faltantes.
¿Cómo modelamos el estado?
 - **Estado:** Nivel de stock de cada producto.
 - **1 Producto:** Si la capacidad es de 100 unidades → **100 estados.**
(Fácil)
 - **3 Productos:** $100 \times 100 \times 100 = 10^6 \rightarrow$ **1 millón de estados.** (Al límite de una tabla)
 - **10 Productos:** $100^{10} = 10^{20}$ estados.

El Problema de la Realidad

Hay más estados en este inventario de 10 productos que granos de arena en la Tierra ($\approx 10^{19}$). **Es imposible llenar esta tabla**, incluso con toda la capacidad de cómputo actual.

Ejemplo: Go



Ejemplo: Go (La Evolución)

Configuración y fuerza⁷

Versiones ↕	Hardwares ↕	Elo ↕	Partidos ↕
AlphaGo Fan	176 GPUs, distribuido	3.144	5:0 contra Fan Hui
AlphaGo Lee	48 TPUs, distribuido	3.739	4:1 contra Lee Sedol
AlphaGo Master	Una sola máquina con 4 TPU v2	4.858	60:0 contra jugadores profesionales; Cumbre del Futuro de Go
AlphaGo Zero	Una sola máquina con 4 TPUs ⁸ v2	5.185 ⁹	100:0 contra AlphaGo Lee 89:11 contra AlphaGo Master

Ejemplo: Go - ¿Cuántos estados?

- Tablero de 19×19
 - $19 \times 19 = 361$ variables de estado (posiciones).
 - Cada posición tiene 3 valores: {blanco, negro, vacío}.
 - **Crecimiento Exponencial:** 3^{361} combinaciones totales.
- Determinar cuántas de esas configuraciones son *válidas* según las reglas del Go es, en sí mismo, un problema masivo.
 - En 2015, John Tromp requirió **9 meses de procesamiento** en computadores de alta potencia para hallar el número exacto:
208168199381979984699478633344862770286522453884
530548425639456820927419612738015378525648451698
519643907259916015628128546089888314427129715319
317557736620397247064859013377169378226061371114
670302056198056 $\approx 2 \times 10^{170}$ configuraciones válidas

Conclusión

Si solo **contar** los estados toma meses de supercómputo, una tabla de Q-Learning es físicamente inviable.

La “Maldición de la Dimensionalidad”

- **Causa Raíz:** El número de estados crece **exponencialmente** respecto al número de variables (C^N).
- **Inviabilidad Tabular:** En problemas reales (Go, Inventarios, Finanzas), el espacio de búsqueda es tan vasto que no se puede visitar, calcular ni almacenar.

Ejemplo: Control de Inventarios (10 productos)

Si cada producto tiene 100 niveles de stock:

- Estados posibles: $100^{10} = 10^{20}$ (más que granos de arena en la Tierra).
- **El error de la tabla:** Trata "99 unidades" como algo totalmente distinto a "100 unidades". No hay transferencia de conocimiento.

Si el espacio de estados es prohibitivo, la memoria debe ser reemplazada por la intuición.

De la Memoria a la Generalización

- **Solución:** Sustituir la tabla por una **forma paramétrica** $f(x, \theta)$.
- **Aproximación de Funciones:** Pasamos de estimar 10^{20} celdas a estimar un vector de pesos $\theta \in \mathbb{R}^K$ (donde $K \ll$ estados).

$$V^*(x) \approx f(x, \theta_V) \quad ; \quad Q^*(x, a) \approx f(x, a, \theta_Q)$$

- **Generalización (Ventaja Crítica):** Al aprender sobre un estado visitado, el modelo "entiende" la tendencia para estados similares no visitados.

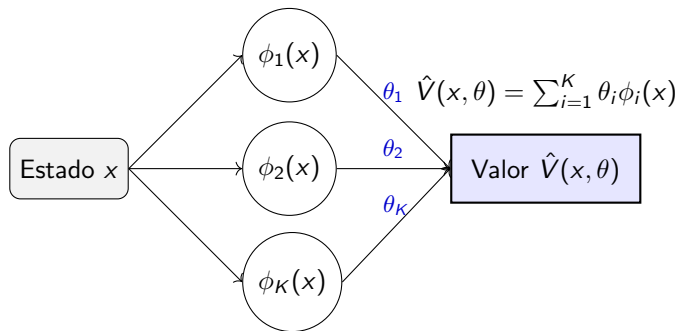
Cambio de Paradigma

Ya no buscamos el valor exacto de cada casilla, sino los **pesos** θ que mejor describen la **estructura** del problema.

- *Nota:* También podemos parametrizar la política $u(x, a, \theta_u)$ como una distribución de probabilidad.

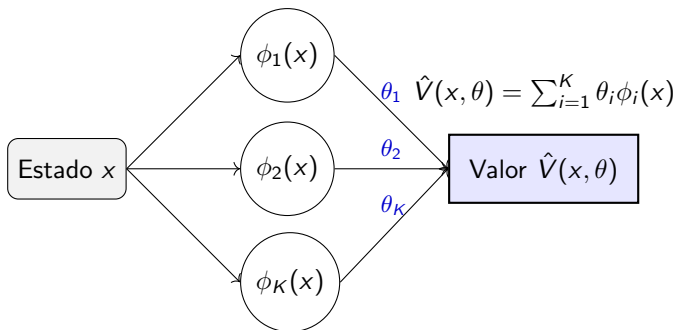
Aproximaciones Lineales

- Queremos aproximar el valor de un estado $V(x)$ sin usar una tabla
- Usamos una combinación de funciones conocidas ("funciones básicas"):



- Esto se llama **aproximación lineal en los parámetros** (aunque $f_i(x)$ puede ser no lineal en x)

Aproximaciones Lineales: Ejemplos

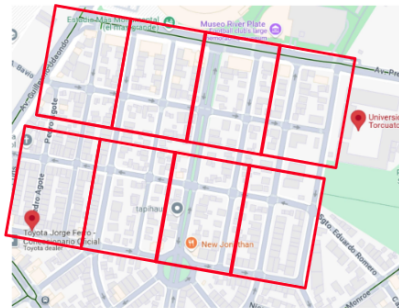


- **Estado directo:** $\phi_i(x) = x_i$.
- **Polinomios:** Combinaciones como $x_1^2, x_1x_2 \dots$ para capturar relaciones.
- **Otras Funciones Base:** Uso de $\sin(x)$, $\log(x)$ o RBFs para mayor flexibilidad.
- **Agregación de estados:** Dividir el espacio en regiones disjuntas.

Agregación de Estados

$$\phi_i(x) = \begin{cases} 1 & \text{si } x \in S_i \\ 0 & \text{si } x \notin S_i \end{cases}$$

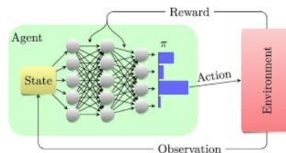
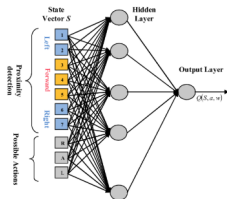
- **Idea:** Agrupar estados similares en zonas.
- El valor aproximado es constante dentro de cada zona: $\hat{V}(x) = \theta_i$.
- **Ventaja:** Simplifica el problema y garantiza estabilidad.
- **Desafío:** Requiere una buena partición del espacio para no perder detalles críticos.



Aproximaciones No Lineales

$$\hat{V}(x, \theta) \approx f(x, \theta)$$

- La función f es **no lineal** respecto a los parámetros θ .
- **Ejemplo principal:** Redes Neuronales Artificiales.
- Permite extraer características automáticamente de datos »crudos« (imágenes, audio).
- Es la base del **Deep Reinforcement Learning**.



Arquitecturas de Aproximación

Categoría	Agregación	Lineal en θ (Manual)	No Lineal en θ (Manual)	No Lineal en θ (Universal)
Expresión	$\sum \theta_i \mathbb{I}_i(x)$	$\sum \theta_i \phi_i(x)$	$f_{\text{expert}}(x, \theta)$	$f_{NN}(x, \theta)$
Representación	Zonas fijas	Features $\phi(x)$ fijas	Estructura fijada por experto	Features aprendidas
Dependencia θ	Lineal	Lineal	No Lineal	No Lineal
Ejemplos	Grillas / Tiles	Polinomios, Fourier	Modelos físicos, Sigmoïdes	Redes Neuronales

Dependencia de Parámetros

La distinción **Lineal** / **No Lineal** define si el valor es una combinación lineal de los parámetros θ , independientemente de x .

- Un modelo »No Lineal Manual« inyecta estructura experta donde θ no actúa como un simple peso.

Comparativa de Convergencia

- **Agregación de Estados:**

- Recupera **garantías de convergencia** del caso tabular.
- Representación rígida (escalonada).

- **Diseño de Features (Lineal en θ):**

- Superficie de error cuadrática respecto a targets fijos.
- El experto guía.^{al} modelo mediante $\phi(x)$.

- **Modelos Estructurales (No Lineal Manual):**

- Inyecta leyes físicas o límites estructurales.
- **No convexo:** Difícil de optimizar pese a pocos parámetros.

- **Aproximadores Universales (Redes):**

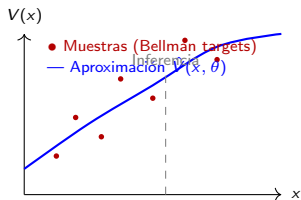
- La red extrae sus propias features (*Representation Learning*).
- Máxima flexibilidad; mayor riesgo de inestabilidad.

Criterio de Selección

Manual: Si conoces la forma funcional de la relación.

Universal: Si el problema es una caja negra compleja.

RL con Aproximación: El Residuo de Bellman



Generalización: Estimamos $V(x)$ para estados no visitados basándonos en la estructura de θ .

¿Es Aprendizaje Supervisado?

- **Similitud:** Ajustamos θ para que $f(x, \theta) \approx y$ mediante $\min_{\theta} \sum (y - f(x, \theta))^2$.
- **Diferencia:** En RL **no existe** un label y externo.
- **Solución:** El valor está definido por la **Ecuación de Bellman**:

$$V^*(x) = \max_a \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V^*(y) \right\}$$

Objetivo: Minimización del Residuo de Bellman

Buscamos los pesos θ que minimizan la discrepancia recursiva del sistema:

$$\min_{\theta} \sum_x p(x) \left[\underbrace{\max_a \left\{ r_a(x) + \alpha \sum_y P_a(x, y) V(y, \theta) \right\}}_{\text{Target calculado por Bootstrapping}} - V(x, \theta) \right]^2$$

Objetivo: Minimización del Residuo de Bellman

$$\min_{\theta} \sum_x p(x) \left[\underbrace{\max_a \{ r_a(x) + \alpha \sum_y P_a(x, y) V(y, \theta) \}}_{\text{Target calculado por Bootstrapping}} - V(x, \theta) \right]^2$$

En la práctica, el espacio de estados es inabarcable y no siempre conocemos las probabilidades de transición $P(x, y)$. No podemos calcular la suma sobre todos los x ni la esperanza sobre todos los y .

- **La Solución: Muestreo de Monte Carlo.** Sustituimos el valor esperado por muestras reales de la experiencia:

$$\text{Error} \approx \sum_{(x_i, r_i, y_i) \in \mathcal{D}} \left[\underbrace{(r_i + \alpha \hat{V}(y_i, \theta))}_{\text{Target (estimación)}} - \underbrace{\hat{V}(x_i, \theta)}_{\text{Predicción}} \right]^2$$

- **Iteración:** Ajustamos θ mediante descenso de gradiente para que nuestra predicción actual se acerque al target calculado con la recompensa observada.

Algoritmos Modernos y Desafíos

Pasar de tablas a funciones (especialmente redes neuronales) introduce inestabilidad.

- **DQN (Deep Q-Networks):** Usa una red neuronal para representar $Q(s, a)$. Para evitar que el algoritmo diverja, se usan:
 - **Replay Buffer:** Recordar experiencias pasadas para romper la correlación de los datos.
 - **Target Networks:** Mantener una copia fija de los parámetros temporalmente para que el "target" no se mueva mientras aprendemos.
- **MuZero / AlphaZero:** Llevan la aproximación al límite, aprendiendo no solo el valor, sino también un modelo interno de las reglas del mundo.

Realidad Técnica

A diferencia del caso tabular, aquí la convergencia **no está garantizada**. Los resultados son muy sensibles a la arquitectura y a los "hiperparámetros" (tasa de aprendizaje, tamaño de red).

Valuación de Opciones Americanas

Una opción es un derivado que otorga el **derecho**, pero no la obligación, de comprar o vender un activo a un precio K (strike) hasta un vencimiento T .

- **Call:** Derecho a comprar el activo subyacente.
- **Put:** Derecho a vender el activo subyacente.

El factor de flexibilidad

- **Europeas:** El derecho se ejerce **solo** en $t = T$.
- **Americanas:** El derecho se puede ejercer en **cualquier momento** $t \in [0, T]$.

El **payoff** de una Put en el momento del ejercicio t es: $\max(K - S_t, 0)$.

¿Cuánto vale una opción americana?

El objetivo principal es determinar el precio justo (V_0) del contrato hoy.

- En una **opción europea**, el valor es simplemente el promedio descontado del payoff final bajo una medida de riesgo neutro.
- En una **opción americana**, la flexibilidad encarece el contrato, pero introduce una **incertidumbre fundamental**:

La Incógnita

No podemos valorar la opción sin saber **cuándo** se va a cobrar el payoff. El flujo de caja no es fijo en el tiempo, sino que depende de la estrategia de ejercicio del poseedor de la opción.

Valuación como Consecuencia de la Decisión

Para hallar el valor de la opción, debemos asumir que el inversor actuará de forma racional para maximizar su beneficio.

- **Dependencia Estratégica:** El valor de la opción hoy es el resultado de la mejor estrategia posible de ejercicio en el futuro.
- **El Dilema Recurrente:** En cada instante t , el valor de la opción es:

$$V_t = \text{máx}(\text{Valor de Ejercicio Inmediato}, \text{Valor de Continuación})$$

- **Valor de Continuación:** Lo que espero ganar si mantengo la opción abierta.
- **Conclusión:** La valuación es, en esencia, un problema de **parada óptima** (*optimal stopping*).

Vínculo Teórico

Valuar una opción americana equivale a encontrar la **política óptima de ejercicio**. Representando el tiempo de parada como τ (*stopping time*), el cual es una **variable aleatoria** que depende de la evolución del precio, el valor es:

$$V_0 = \sup_{\tau \in [0, T]} \mathbb{E}[e^{-r\tau} \text{Payoff}(S_\tau)]$$

Este problema de parada óptima encaja en el marco de los Procesos de Decisión de Markov (MDP). Para resolverlo, debemos definir formalmente sus componentes: estados, acciones, recompensas y transiciones.

La Opción Americana como un MDP Completo

Consolidamos el modelo (S, A, P, R, α) para una Put Americana:

- **Estado (s_t):** El precio del activo subyacente en el periodo t .
- **Acción (a_t):**
 - $a = 0$: **Continuar** (mantener la opción abierta).
 - $a = 1$: **Ejercer** (cobrar payoff y terminar episodio).
- **Recompensa (R):**
 - $R(s, a = 0) = 0$
 - $R(s, a = 1) = \max(K - s, 0)$
- **Factor de Descuento:** $\alpha = e^{-r_f \Delta t}$.
- **Transiciones (P):** Debemos modelar la evolución del precio del activo.

Dinámica de Precios para Valuación de Opciones

Para valorar derivados, se modela la evolución de precios bajo la **Medida de Riesgo Neutro**, uno de los pilares de las finanzas cuantitativas.

Modelo Lognormal de Precios

Bajo esta medida, la evolución del precio del activo entre periodos es:

$$S_{t+1} = S_t e^{(r_f - \frac{\sigma^2}{2})\Delta t + \sigma\sqrt{\Delta t}\epsilon_t} \quad \text{donde } \epsilon_t \sim \mathcal{N}(0, 1)$$

- **Intuición:** En un mundo ideal donde los inversores no exigen una prima por correr riesgos, todos los activos financieros deberían rendir, en promedio, la tasa libre de riesgo (r_f).
- **Implicancia:** Esto nos permite valorar opciones simplemente descontando sus beneficios esperados usando r_f , sin preocuparnos por las preferencias individuales frente al riesgo.

Medida de Riesgo Neutro: Un Resultado Fundamental

- **Modelo para Optimización de Portafolios (Mundo Real):**

$$s_{t+1} = s_t e^{(\mu - \frac{\sigma^2}{2})\Delta t + \sigma\sqrt{\Delta t}\epsilon_t}$$

Donde μ es el rendimiento esperado que incluye una prima por riesgo.

- **Modelo para Valuación (Mundo Riesgo-Neutro):**

$$s_{t+1} = s_t e^{(r_f - \frac{\sigma^2}{2})\Delta t + \sigma\sqrt{\Delta t}\epsilon_t}$$

Donde μ es reemplazado por r_f (tasa libre de riesgo).

Importancia del Resultado (Black-Scholes-Merton)

Este es uno de los mayores hitos de las finanzas: el precio de una opción **no depende del rendimiento esperado del activo (μ)**.

- No importa si el inversor es optimista o pesimista sobre el futuro del activo; si acuerdan la volatilidad (σ), llegarán al mismo precio justo.
- El riesgo se “cancela” mediante la posibilidad de replicar la opción operando el activo y bonos.

La Opción Americana como un MDP

Consolidamos el modelo (S, A, P, R, α) para una Put Americana:

- **Estado (s_t):** El precio del activo subyacente en el periodo t .
- **Acción (a_t):**
 - $a = 0$: **Continuar** (mantener la opción abierta).
 - $a = 1$: **Ejercer** (cobrar payoff y terminar episodio).
- **Recompensa (R):**
 - $R(s, a = 0) = 0$
 - $R(s, a = 1) = \max(K - s, 0)$
- **Factor de Descuento:** $\alpha = e^{-r_f \Delta t}$.
- **Transiciones (P):** Si $a = 0$, el estado evoluciona según la dinámica de riesgo neutro:

$$s_{t+1} = s_t e^{(r_f - \frac{\sigma^2}{2})\Delta t + \sigma\sqrt{\Delta t}\epsilon_t}$$

Este marco nos permite usar RL para encontrar la política de parada óptima y precificar la opción.

Ecuaciones de Bellman: Continuar vs. Ejercer

Para valorar la opción, analizamos la función de valor óptima $Q^*(s, a)$. En cada periodo t , el poseedor enfrenta una decisión binaria:

- **Valor de Ejercicio ($a = 1$):** Es el payoff inmediato, conocido para cualquier t :

$$Q_t^*(s, 1) = K - s$$

- **Valor de Continuación ($a = 0$):** Es la expectativa del valor de la opción en el próximo periodo, descontada al presente:

$$Q_t^*(s, 0) = \alpha \mathbb{E} [\text{máx} (Q_{t+1}^*(s_{t+1}, 0), K - s_{t+1}) \mid s_t = s]$$

- **Condición de Borde Final:** En el vencimiento ($t = T$), si no se ejerce, la opción expira sin valor:

$$Q_T^*(s, 0) = 0$$

Problemas de Parada Óptima y RL

La valuación de opciones americanas pertenece a la clase de MDPs conocidos como **problemas de parada óptima**. El objetivo es aprender una política que decida el momento exacto τ para detener el proceso y capturar el valor.

- **Múltiples Enfoques de RL:**

- **Métodos de Valor (Value-based):** Aproximar $Q_t^*(s, a)$ mediante arquitecturas lineales o redes neuronales (DQN).
- **Programación Dinámica Aproximada (ADP):** Resolver Bellman hacia atrás usando regresión para estimar el valor de continuación.
- **Policy Search:** Optimizar directamente los parámetros de una frontera de ejercicio (ej. buscar el umbral óptimo).

- **Desafío del Estado Continuo:** Dado que el precio s_t es continuo, no podemos usar tablas. Necesitamos una arquitectura de aproximación. Hoy vamos a utilizar una arquitectura lineal:

$$Q_t^*(s, 0) \approx \Phi(s, \theta_t) = \sum_{j=1}^K \phi^j(s) \theta_t^j$$

Fitted Q-Iteration (FQI)

FQI es un algoritmo de **Aprendizaje por Refuerzo Offline** que convierte el problema de control en una secuencia de problemas de aprendizaje supervisado.

- **Objetivo:** Aproximar la función de valor óptima $Q^*(s, a)$ cuando el espacio de estados es continuo.
- **Mecánica:** En lugar de una tabla, usamos un aproximador de funciones (en nuestro caso, regresión lineal):

$$Q(s, a; \theta) \approx \Phi(s, a)^T \theta$$

- **Iteración:** Se actualizan los parámetros θ aplicando el operador de Bellman sobre un conjunto de datos (trayectorias simuladas).
- Para valuación de opciones, es suficiente estimar $Q(s, 0; \theta) \approx \Phi(s)\theta$, el valor de esperar ($a = 0$).

FQI: El Caso Base ($t = T-1$)

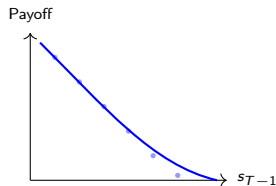
En el penúltimo periodo, valuamos la opción como una regresión lineal estándar. El target es el payoff real observado en el vencimiento.

$$\min_{\theta_{T-1}} \sum_{i=1}^N [\Phi(s_{T-1}^i, \theta) - y_{T-1}^i]^2$$

donde el target es:

$$y_{T-1}^i = \alpha \max(K - s_{T-1}^i, 0)$$

- X: Features $\phi(s_{T-1})$.
- y: Payoff en T descontado.



Recursión: Definiendo el Target ($t < T - 1$)

Para periodos anteriores, seguimos usando regresión lineal, pero ¿cómo elegimos el Target (y_t^i)?

Hay distintas opciones, por ejemplo...

1. Target FQI (Bellman / Q-Learning)

$$y_t^i = \alpha \cdot \text{máx} (\text{Payoff}(s_{t+1}^i), \Phi(s_{t+1}^i, \theta_{t+1}))$$

Usa la propia aproximación para estimar el futuro (Bootstrapping).

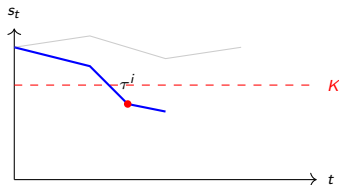
2. Target LSM (Longstaff-Schwartz)

$$y_t^i = \alpha \cdot \text{Payoff}(s_{\tau_i}^i)$$

Usa el payoff real cobrado en la trayectoria según la política previa (Monte Carlo).

FQI Parte II: Valuación (Forward Simulation)

No usamos Φ directamente para el precio final. Simulamos **nuevas trayectorias** y aplicamos la política aprendida.



- ① **Stopping Time (τ^j):** Primer t donde ejercer es mejor que el valor de continuación predicho:

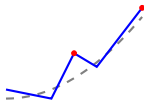
$$\tau^j = \min\{t : K - s_t^j > \Phi(s_t^j, \theta_t)\}$$

- ② **Precio:** $\hat{V}_0 = \frac{1}{M} \sum_{j=1}^M \alpha^{\tau^j} \text{Payoff}(s_{\tau^j}^j)$

Análisis Crítico: El Sesgo de Maximización

¿Por qué $\Phi(s_0, \theta_0)$ suele ser mayor que el precio por simulación?

- **Maximization Bias:** El operador máx elige sistemáticamente errores positivos de la aproximación ruidosa.
- **Consecuencia:** FQI tiende a sobreestimar el valor de la opción en el entrenamiento.



Desafíos y Complejidad del MDP

Aunque el modelo de una variable (s_t) es didáctico, los problemas reales enfrentan desafíos adicionales:

- **Alta Dimensionalidad:**

- Modelos con volatilidad estocástica (σ_t) o tasas de interés dinámicas (r_t).
- *Basket options*: Opciones que dependen de múltiples activos simultáneamente.

- **La "Maldición de la Dimensionalidad"**: El número de funciones base necesarias para la aproximación lineal crece exponencialmente con cada nueva variable de estado.

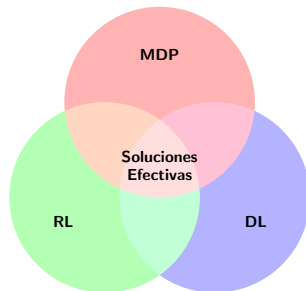
- **Continuidad**: Siempre trabajaremos con una aproximación; la precisión depende de la riqueza de nuestras funciones base (ϕ^j).

De LSM a Deep RL

Cuando la dimensionalidad es muy alta, sustituimos la regresión lineal por **Redes Neuronales Profundas** para aproximar el valor de continuación.

Conclusiones: El Futuro del Control y la Decisión

- **Flexibilidad:** El paradigma MDP modela desde finanzas hasta robótica y logística.
- **Sinergia:** La combinación de MDP con *Machine Learning* permite soluciones competitivas en entornos inciertos.
- **Escalabilidad:** Las aproximaciones de valor y política son la llave para la alta dimensionalidad.
- **Pragmatismo:** La complejidad del algoritmo debe ser coherente con la calidad de los datos disponibles.
- **Impacto:** Las Redes Neuronales (Deep RL) abren fronteras antes inalcanzables.



Desafío

La implementación correcta y estable sigue siendo el mayor reto del ingeniero.